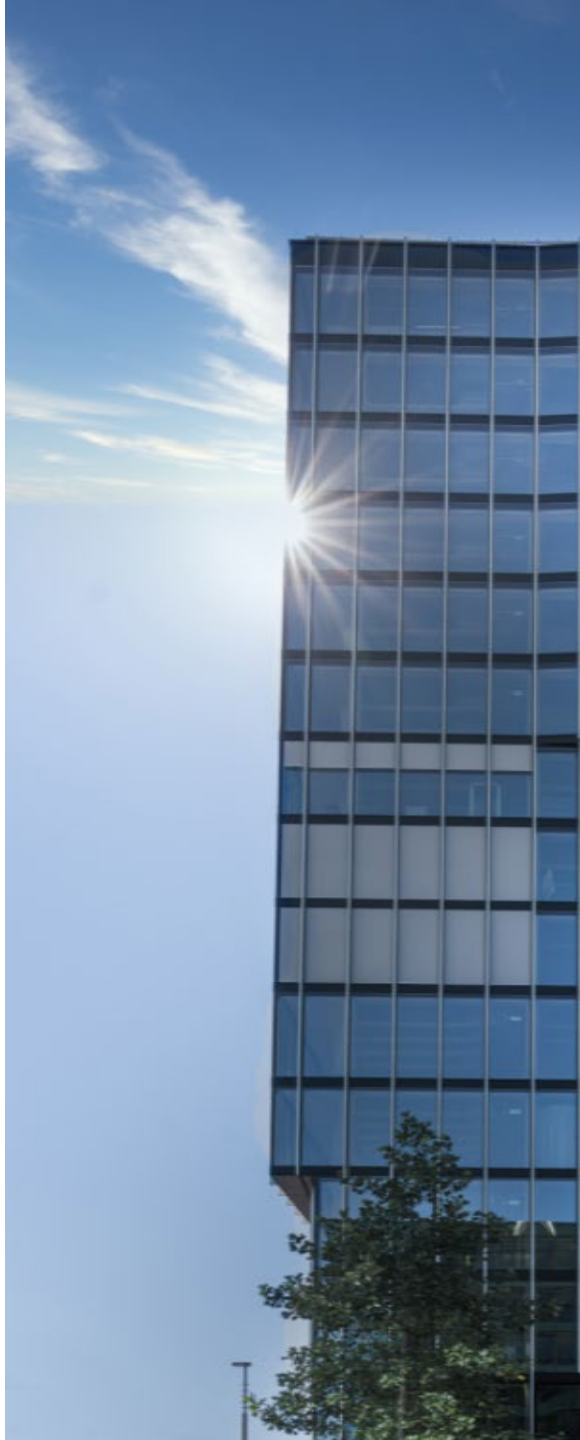
Objektorientierte Programmierung

Entwicklungsumgebungen

IDE – Integrated Development Environment

Roland Gisler



Inhalt

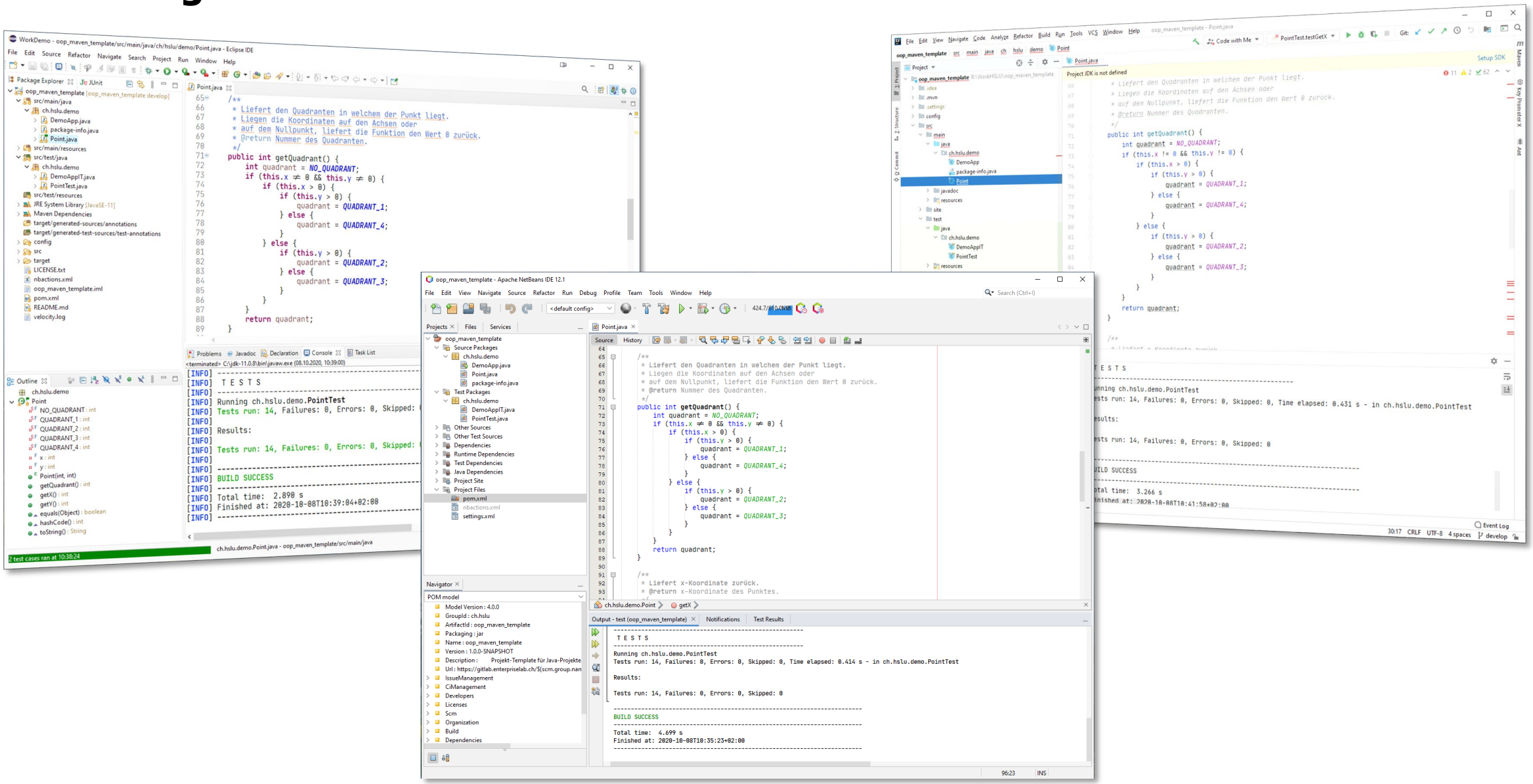
- Was ist eine IDE und was sind ihre Aufgaben?
- Wie sind IDEs eigentlich entstanden?
- Aktuelle IDEs für Java
- Einsatz und Nutzung der IDE in OOP
- IDE versus BlueJ
- Empfehlungen zu IDEs
- Empfehlungen zum Einsatz in OOP

Lernziele

- Sie wissen, was eine IDE ist und welchem Zweck sie dient.
- Sie kennen die wichtigsten Aufgaben einer IDE.
- Sie können mit einer IDE grundlegende Programmiertask erledigen.
- Sie kennen eine für Java-Projekte adäquate Projekt- bzw. Verzeichnisstruktur.

Was ist eine IDE?

Was ist eigentlich eine IDE*?



*IDE = **I**ntegrated **D**evelopment **E**nvironment

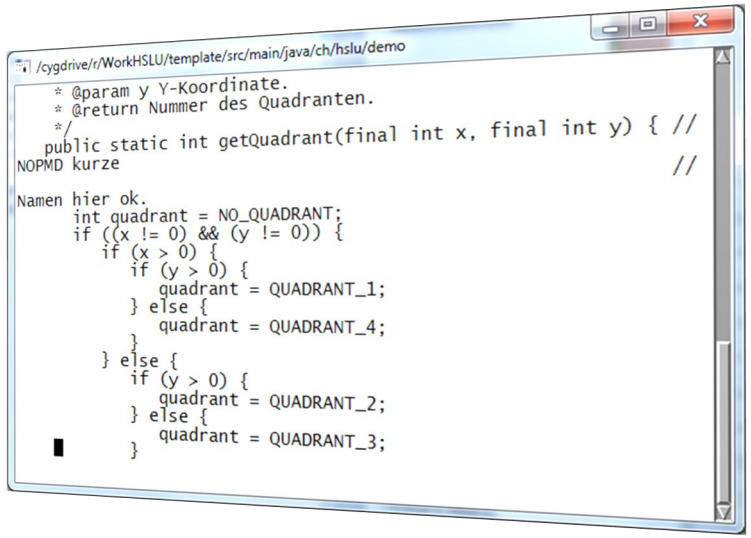
Die wichtigsten Aufgaben einer IDE

- Visualisierung und Navigation von/im Quellcode.
- Leistungsfähige Quellcode-Editoren.
- Werkzeuge für Refactoring.
- Organisation von Quellcode in Projekten.
- Organisation der Konfiguration von Projekten.
- Quellcode kompilieren und ausführen, Debugging.
- Anbindung an Versionskontrollsysteme.
- Erzeugung, Import und Export von verschiedenen Artefakten.
- Und vieles, vieles mehr!

Wie sind IDEs entstanden?

Wie haben sich die IDEs entwickelt?

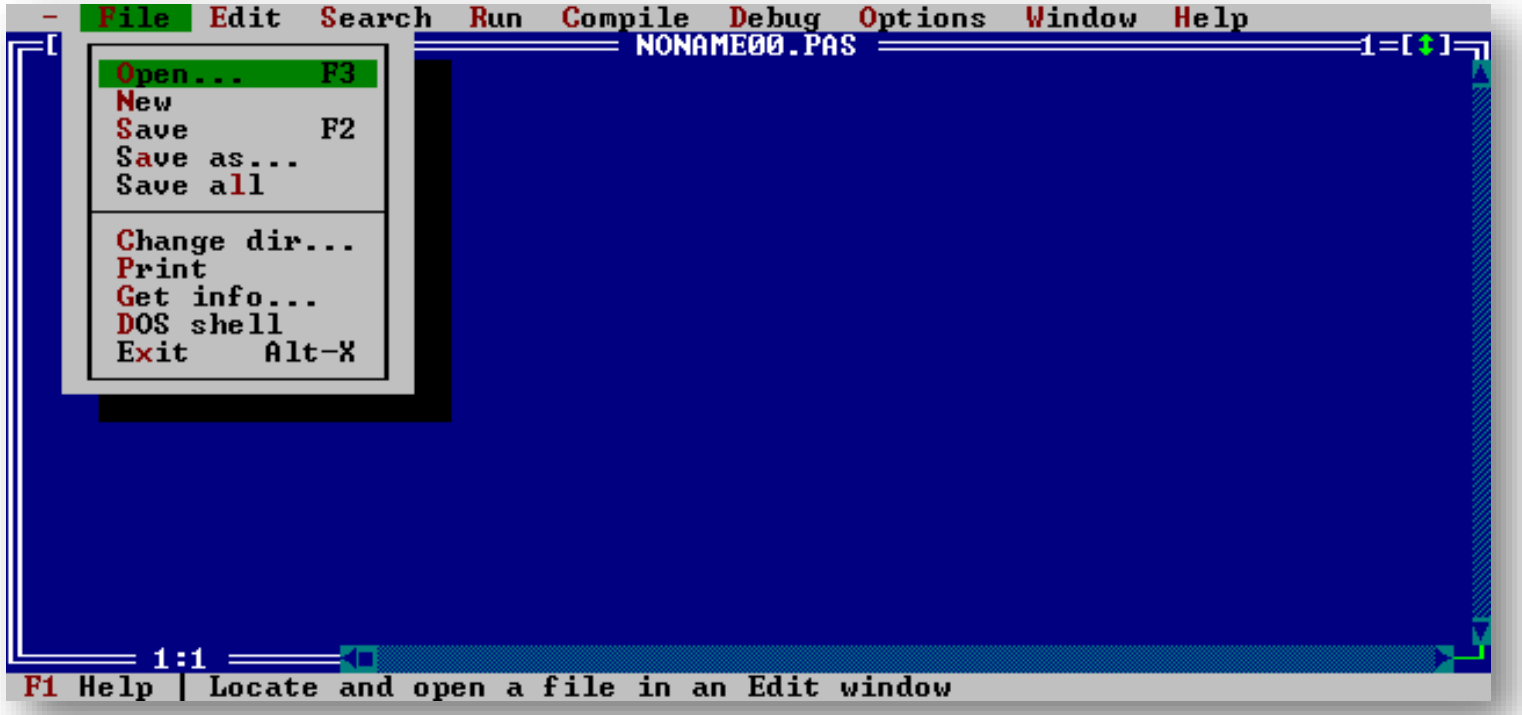
- Ursprünglich wurde Code mit normalen Editoren editiert…
  - z.B. unter Unix/Linux: **`vi, vim, emacs`**
- mit einem Kompiler-Programm kompiliert…
  - z.B. für Java: **`javac <java-Datei>`**
- und zu einer Library verlinkt / verpackt.
  - z.B. für Java: **`jar <jar-Datei> <class-Dateien…>`**



- Ein fehleranfälliger Vorgang mit grossem Overhead!
  - Editieren, Kompilieren, Testen – Editieren, Kompilieren…
- Editoren (z.B. **`emacs`**) konnten funktional erweitert werden, so dass z.B. die Kompilation innerhalb des Editors gestartet werden konnte.
  - **Integration** der Entwicklungs-Tools ➔ die Geburt der **IDE**!

Populäre frühe IDEs

- z.B. Borland Turbo Pascal und Turbo C/C++



- DOS/konsolenbasierendes Programm – Ende der 1980er-Jahre
- Editor mit integriertem Compiler und Debugger

  ➔ Geburt der IDE im heutigen Sinne!

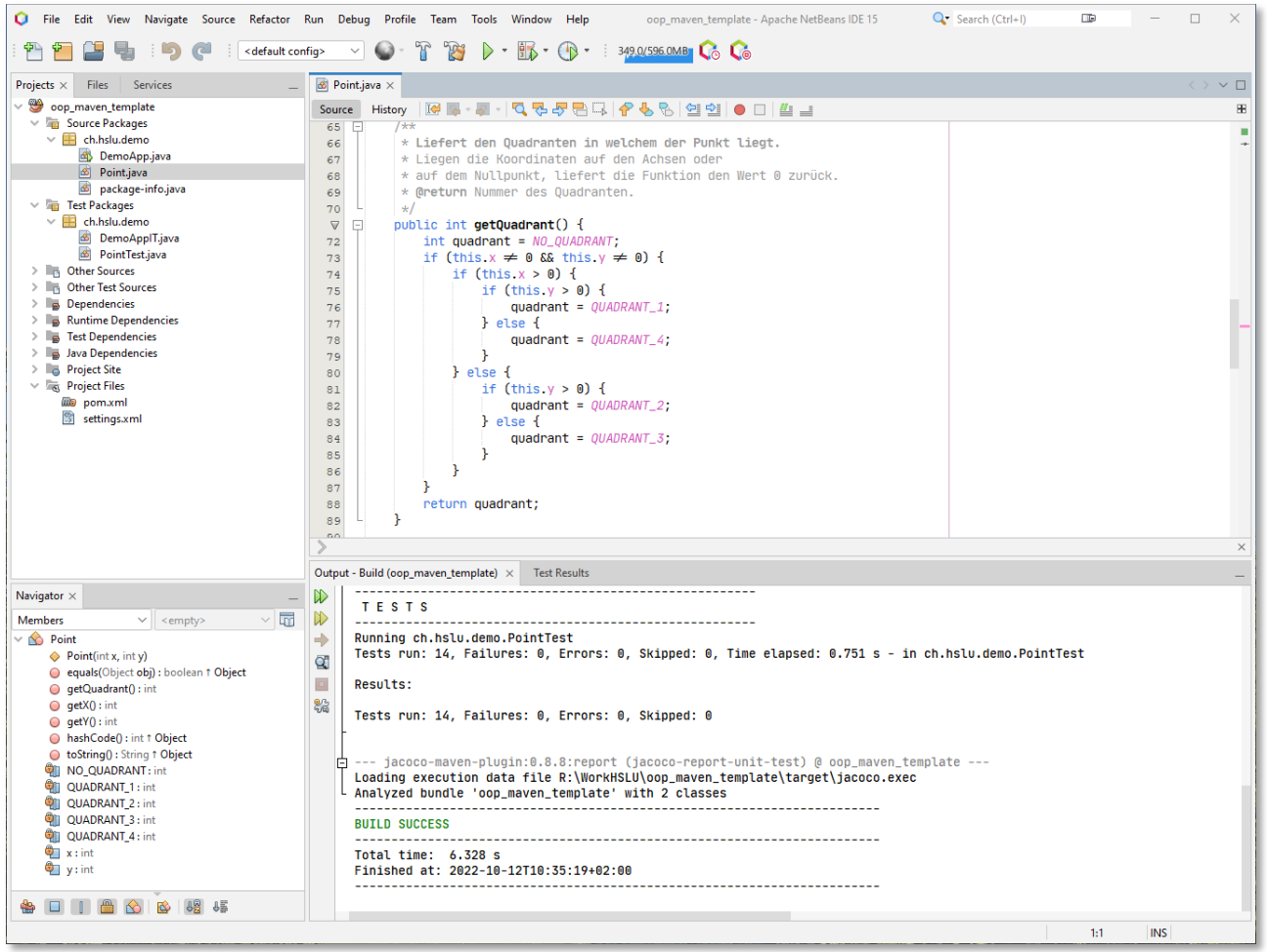
Aktuelle IDEs für Java

Aktuelle IDEs / Beispiele

- IDEs haben sich immer mehr und weiter zu «integrierenden» Entwicklungs-umgebungen verändert.
- Die flexiblen Plugin-Konzepte erlauben:
  - Eine nahezu beliebige Erweiterung der Funktionalität.
  - Integration oder Verknüpfung mit anderen Systemen.
    - z.B. SCM, Datenbanken, Buildserver, Issue-Tracking etc.
- Beispiele von populären (primär Java-)IDEs:
  - **Apache NetBeans**: Open Source, eine der ältesten Java-IDEs.
  - **VSCodium** (oder **Visual Studio Code**): Kann auch für Java genutzt werden.
  - **IntelliJ IDEA**: Kommerzielle IDE von JetBrains, Community Edition gratis, sehr leistungsfähig, für diverse Sprachen.
  - **Eclipse JDT**: Open Source, ursprünglich nur Java, heute auch für C/C++, PHP und viele andere Sprachen.

Apache NetBeans IDE

- Die Altbewährte - http://netbeans.apache.org/
- Aktuelle Version: **29**



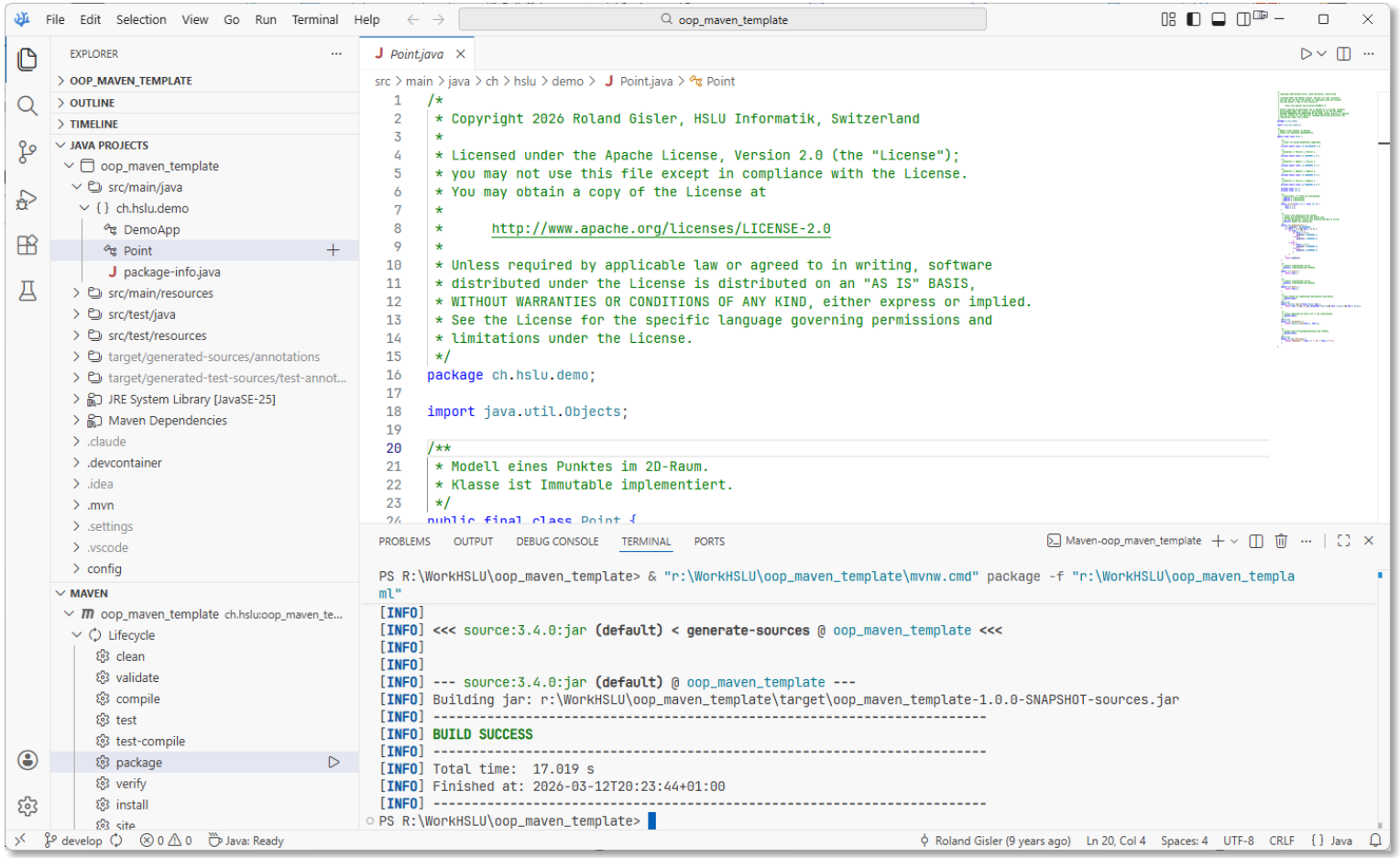
Apache NetBeans IDE - Hinweise

- Perfekte Integration der auf Apache Maven basierten Projekte.
- «Klassische» Entwicklungsumgebung, einfaches GUI, relativ übersichtlich und einfach in der Bedienung.
- IDE merkt sich globale Einstellungen zentral für alle Projekte.
- Projekte können einfach «geöffnet» und «geschlossen» werden.
- Es ist problemlos möglich, auch mehrere Projekte parallel geöffnet zu halten.
- Eine der wenigen Entwicklungsumgebungen, die nicht KI-verseucht ist.

- **Empfehlung**: Studierende, welche noch keine Entwicklungsumgebung gut kennen, und/oder sich neu mit Programmierung beschäftigen und die bezüglich KI auf der sicheren Seite sein wollen: **NetBeans ist Ihre erste Wahl!**

VSCodium

- Die freie Open Source Variante von Visual Studio Code – https://vscodium.com/
- Aktuelle Version: **1.110.x**



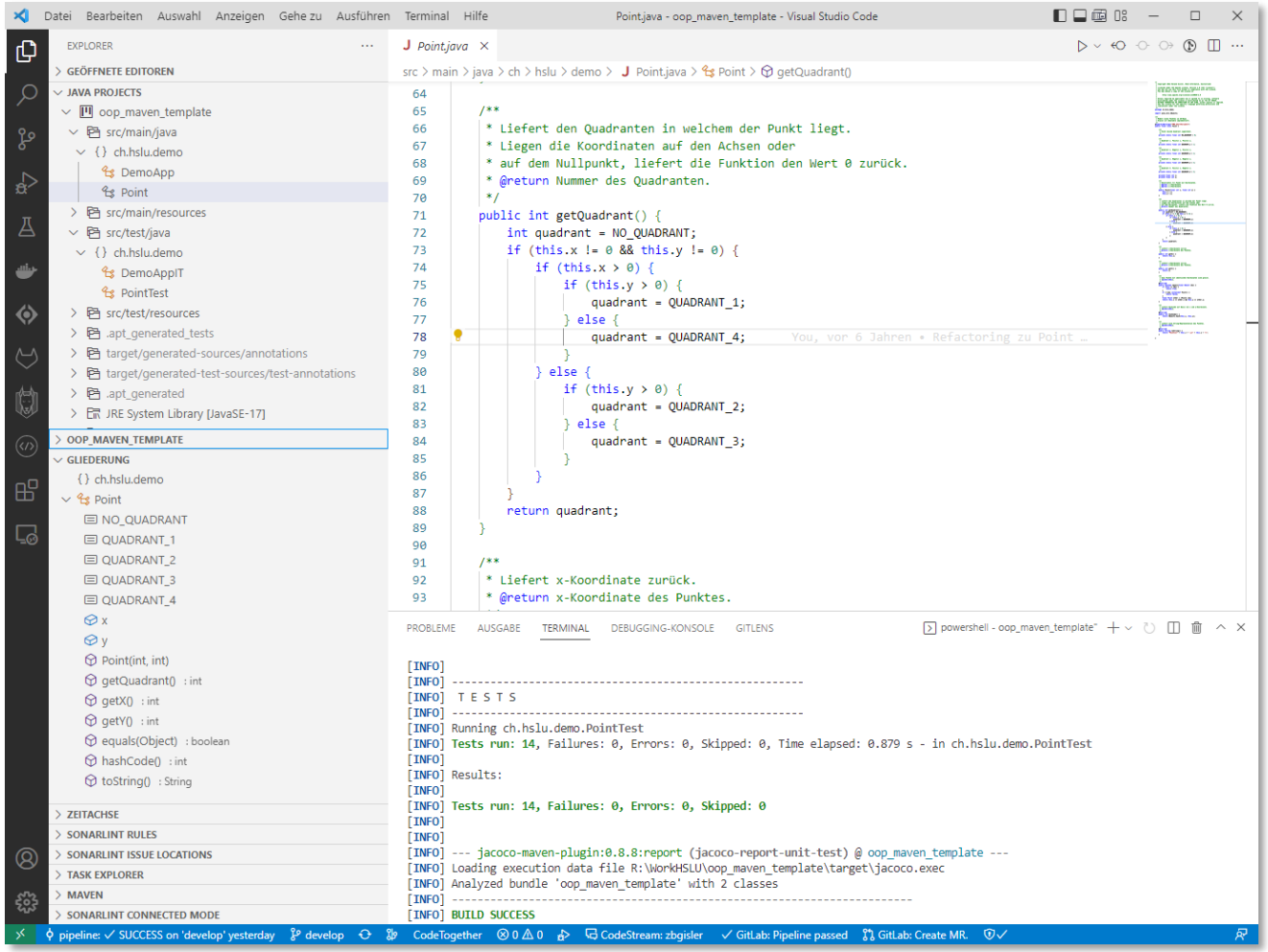
VSCodium– Hinweise

- Ursprünglich keine Java IDE, sondern für C# ausgelegt.
- VSCodium stellt die Basis für Visual Studio Code dar, und ist entsprechend gut in Windows eingebettet, macht das aber nicht so «aggressiv» wie Visual Studio Code.
  - Im Moment das «lebendigste» Ökosystem, was die Plugin-Entwicklung betrifft.
- Kann mittels Plugins mit sehr vielen verschiedenen Programmiersprachen umgehen. Diese müssen aber installiert werden.
- Die Java-Plugins sind ausgereift und basieren auf Eclipse-Technologien: «**Extension Pack for Java**» von «**vscjava**» - das reicht!

- **Empfehlung**: Wenn Sie Visual Studio Code schon beherrschen (nicht nur kennen), probieren Sie doch mal **VSCodium** aus! Oder nutzen Sie alternativ mal ein «echte», reine Java-IDE wie NetBeans IDE!

Visual Studio Code

- Bei Windows sehr verbreitet – https://code.visualstudio.com/
- Aktuelle Version: **1.110.x**



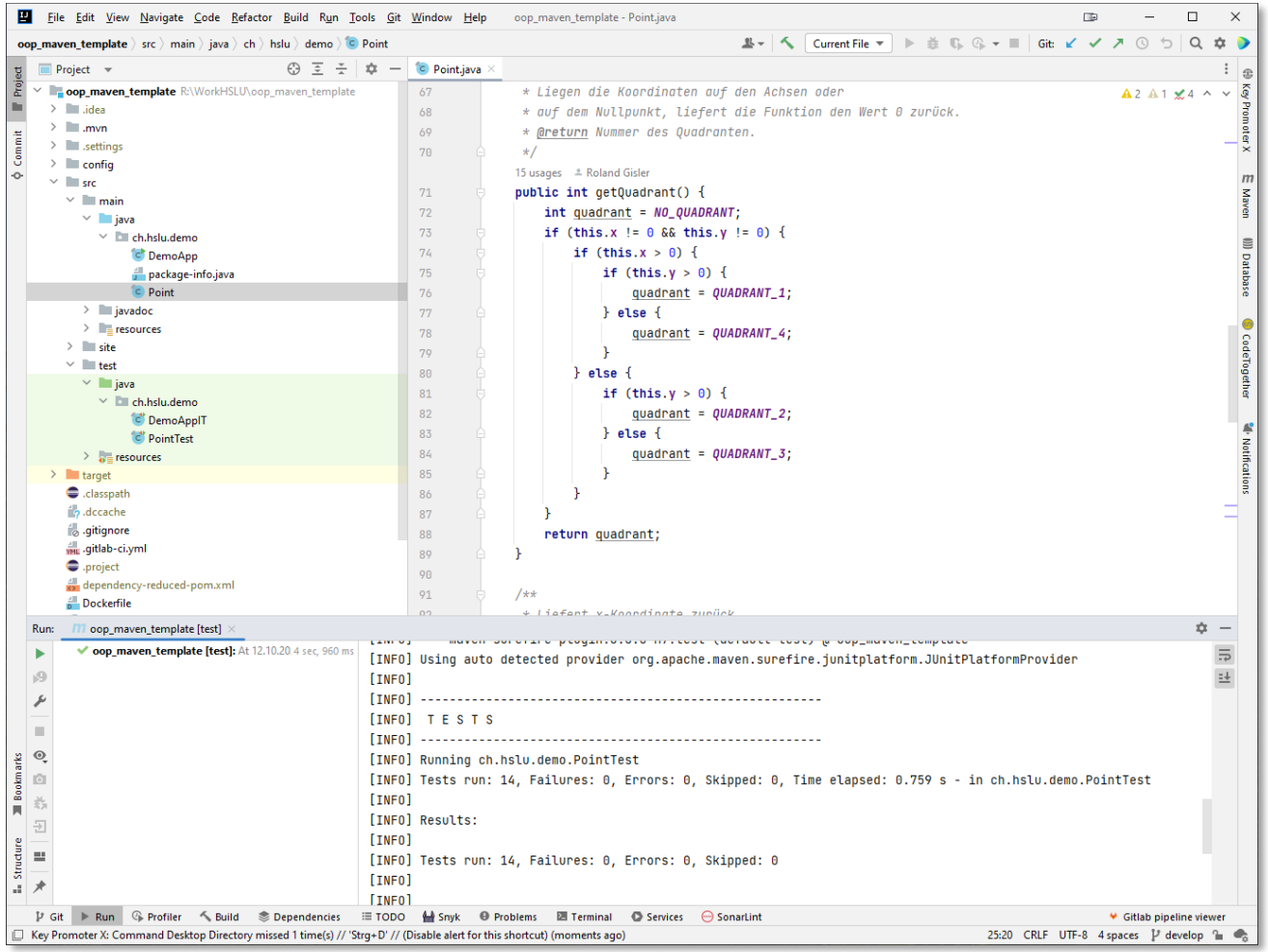
Visual Studio Code – Hinweise

- Basiert auf VSCodium, Details siehe darum siehe VSCodium!
- Entwicklung wir von Microsoft getrieben, stärkere Integration in/mit Betriebssystem und aggressiver Ausbau mit KI-Fähigkeiten.
  - KI-Funktionalität sehr aufdringlich. Für den Lernprozess aber **nicht** förderlich.
  - Überfordert weniger erfahrene Studierende mit vielen Tipps und Hinweisen.

- **Empfehlung**: Wenn Sie Visual Studio Code schon beherrschen (nicht nur kennen), probieren Sie für OOP doch mal VSCodium aus!

IntelliJ IDEA – «Community Edition»

- Unter Java-Kenner*innen sehr beliebt – http://jetbrains.com/idea/
- Aktuelle Version: **2025.3.3**

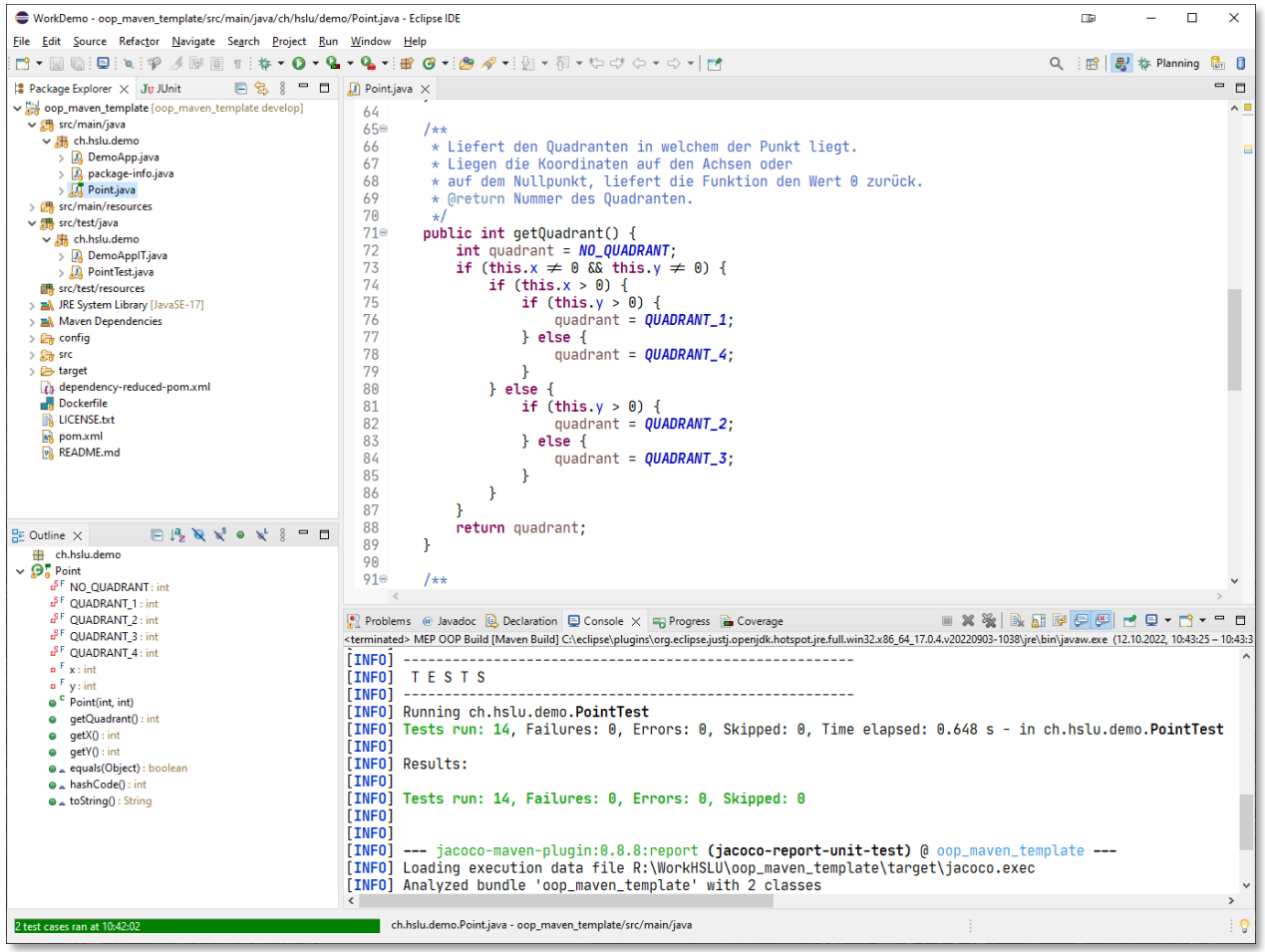


IntelliJ IDEA – Hinweise

- Gute Integration der auf Apache Maven basierten Projekte.
- Konfiguration pro einzelnes Projekt. Mehrere Projekte werden in mehreren IDE-Instanzen geöffnet.
- IDE ist stark auf Effizienz getrimmt, Bedienung ist sehr stark auf Shortcuts ausgelegt.
  - Dazu muss man diese aber kennen, und wissen, was man will!
  - KI-Funktionalität sehr aufdringlich. Für den Lernprozess aber **nicht** förderlich.
  - Überfordert weniger erfahrene Studierende mit vielen Tipps und Hinweisen.
- Zielgruppe liegt klar bei bereits erfahrenen, professionellen Entwickler*innen, die genau wissen, was sie machen, was sie wollen und was sie brauchen.
- **Empfehlung**: Bitte nur verwenden, wenn Sie schon über aktive Entwicklungs-erfahrung verfügen und/oder IntelliJ bereits wirklich kennen. IntelliJ ist **keine** IDE für «Anfänger*innen».

Eclipse JDT – «Eclipse IDE for Java Developers»

- Populär und leistungsfähig – https://www.eclipseide.org/
- Aktuelle Version: **2026-03**



Eclipse JDT – Hinweise

- Gute Integration der auf Apache Maven basierten Projekte.
- Bedienung ebenfalls über ein klassisches Menü möglich.
- Konzept der «Workspaces»: Einstellungen werden pro Basisverzeichnis (kann mehrere Projekte enthalten) gespeichert.
- Mehrere Projekte können gleichzeitig in der IDE erfasst sein, und zusätzlich noch einzeln geöffnet/geschlossen (aktiv/inaktiv) sein.
  - Einfache Gruppierung von Projekten möglich (Project-Sets)
- Flexibles Plugin-Konzept, zusätzlich auch über «Marketplace» und Account über mehrere Installationen synchronisierbar. Sehr viele Plugins verfügbar.

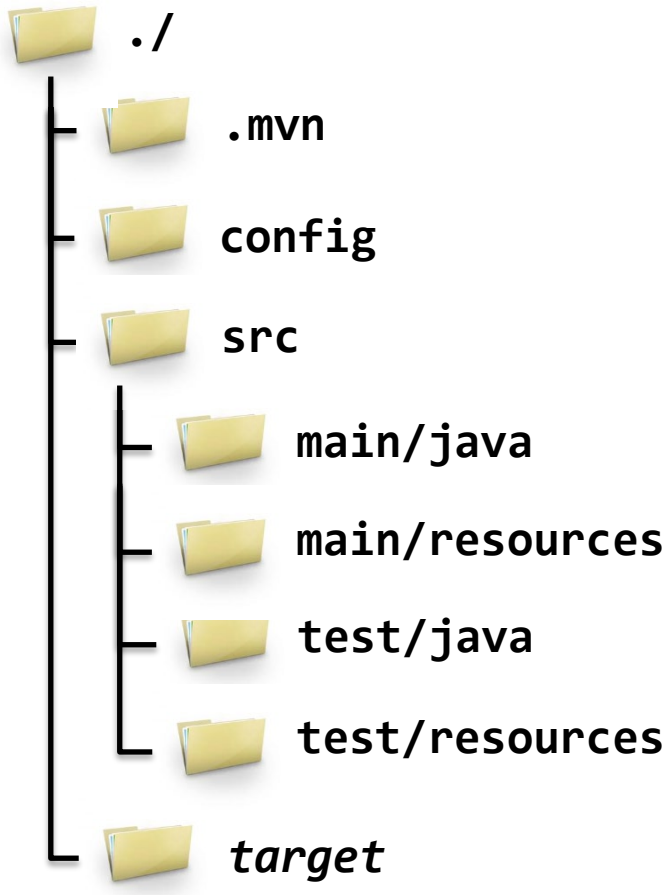
- **Empfehlung**: Wer Eclipse schon kennt, kann dabei bleiben! Es ist eine gute Alternative zu NetBeans und sehr ähnlich in der Bedienung.

Wie arbeiten wir mit der IDE?

Modul OOP - Arbeitsweise mit IDE

- Wir stellen Ihnen für OOP ein vorkonfiguriertes «Template»-Projekt zur Verfügung.
  - Vereinfacht den Einstieg in die professionelle Entwicklung(-sstruktur).
- Dieses Template-Projekt definiert unter anderem eine **Standard-Verzeichnisstruktur** für die Ablage aller Quell-Artefakte.
- Es beinhaltet einen automatisierten, auf ➔Apache Maven basierenden und somit **IDE-unabhängigen Buildprozess**.
  - Kompilierung, Generierung der Javadoc, Ausführen von Testfällen, Messung Codeabdeckung, statische Codeanalyse etc.
- Sie können ganz «normal» in Ihrer IDE arbeiten, die enthaltene Funktionalität ist für Sie noch weitgehend eine «Blackbox».
  - Wird auch in Folgemodulen (AD, VSK, SWDA, DEVOPS etc.) intensiv verwendet und ab dem Modul VSK zunehmend detaillierter erklärt.

Verzeichnisstruktur – Template (Quellen)



- Basisverzeichnis des Projektes.
- Konfiguration von Apache Maven.
- Diverse Konfigurationsdateien (QS).
- Basisverzeichnis für produktiven Java Quellcode.
- Konfigurationen und Ressourcen, für produktive Quellen.
- Basisverzeichnis für Testcode (typisch JUnit Testfälle).
- Konfigurationen und Ressourcen, für Testfälle.
- Zielverzeichnis für alle Buildartefakte, wird generiert, und kann jederzeit gelöscht werden.

Demo – IDEs

Anwendung des Templates - Kurzfassung

- ZIP-Datei im gewünschten Verzeichnis (Workspace) entpacken.
  - Basisverzeichnis für entsprechendes Projekt nach Wunsch umbenennen.
- Projekt in der IDE öffnen bzw. importieren.
  - Projekt (in Maven) umbenennen (Empfehlung: Synchron zum Verzeichnis).
- Sobald Sie eigene Klassen implementiert haben, können Sie die vorhandenen Beispielklassen und Testfälle löschen.
- **Achtung**: Wir entwickeln plattformübergreifend!
  ➔ File-Encoding in Editoren auf **UTF-8** setzen (mittlerweile mehrheitlich Standard)!
  - Damit der Austausch auch über verschiedene Plattformen (Windows, Mac, Linux etc.) funktioniert.
- Mehr und detailliertere Infos finden Sie in der Anleitung auf ILIAS: **OOP_JavaDevelopmentManual_jdk25.pdf**.

IDE vs. BlueJ

Vergleich: Arbeit in BlueJ vs. Entwicklungsumgebung

- **Lern**umgebung - BlueJ:
  - Einfacher Einstieg in die objektorientierte Programmierung.
  - Typisch für sehr kleine, überschaubare und interaktive **Lern**projekte.
  - Quellen und Artefakte liegen in einem einzigen Verzeichnis.
- **Entwicklungs**umgebung - NetBeans, Eclipse JDT, IntelliJ, VSC
  - Für die professionelle Softwareentwicklung ausgelegt.
  - Es ist deutlich mehr automatisiert und «intelligenter».
  - Wir arbeiten mit einer wohldefinierten Verzeichnisstruktur.
  - Wir arbeiten mit Java Packages (Empfehlung: pro Semesterwoche, ggf. Aufgabe)
  - Wir arbeiten in einem **einzigen** Projekt (weniger overhead).
  - Wir ➔testen (ab SW06) wesentlich effizienter automatisiert.

➔ Keine Konkurrenz – da **nicht** wirklich **vergleichbar**!

Empfehlungen zu IDEs

Empfehlungen – sich mit IDE auseinandersetzten

- Setzen Sie sich bewusst mit der ausgewählten IDE auseinander und lernen Sie sie wirklich kennen!
  - Erlernen Sie ganz bewusst die Bedienung der IDE (➔Shortcuts).
- Eine IDE können Sie auch wechseln oder verschiedene parallel benutzen!
  - Machen Sie sich nicht zu stark von einer IDE abhängig!
  - Verschiedene Projekt(-arten) mit verschiedenen IDEs bearbeiten.
- Eine IDE muss Sie in Ihrer Arbeit(-smethodik) unterstützen.
  - Sie soll Ihnen keine «Umwege» aufzwingen und Sie nicht behindern.
- Plugins gehören dazu und machen auch Spass, aber:
  - Die Suche und Auswahl von Plugins ist zeitaufwändig.
  - Die Abhängigkeit wird dadurch wieder grösser / komplexer!
- **Jede** IDE unterstützt ein «dark theme» - somit **kein** Auswahlkriterium!

Empfehlungen – bleiben Sie offen!

- Mit der Zeit (und Erfahrung) wird sich Ihnen eine "Lieblings-IDE" herauskristallisieren.
  - Dennoch (oder gerade deshalb) kann es auch sein, dass Sie in Zukunft parallel mit mehreren IDEs arbeiten werden.
- **Die perfekte IDE gibt es nicht**. Sie werden immer Nachteile und Schwächen finden.
  - Die «beste» IDE ist eine ziemliche Glaubensfrage…
- Wichtig: Wählen Sie die IDE **nicht** nur aufgrund der Optik aus, oder weil sie als «cool» gilt! Verzichten Sie für einen maximalen Lerneffekt vorerst komplett auf KI-Integrationen.
- **Alle** IDE versuchen Ihnen mit zahlreichen Tipps zu helfen. Diese Tipps sind aber für erfahrene Entwickler*innen.
  - Für Unerfahrene führen die Tipps manchmal in die Sackgasse.

Empfehlungen für die Übungen

Empfehlungen - für die Übungen

- Wir empfehlen Ihnen für die Übungen des Modules OOP ein **einziges** Projekt zu erstellen.
  - Beispiel: `oop_exercises`
- Die einzelnen Übungen können Sie darin über **Packages** gliedern.
  - Pro Semesterwoche nummeriert, z.B. `ch.hslu.oop.sw05`
- Enthält eine Übung mehrere Teile, können Sie die Namensgebung individuell weiter verfeinern, z.B. `ch.hslu.oop.sw07.part1`
- **Vorteile** dieses Vorgehens:
  - Sie müssen nicht ständig neue Projekte einrichten.
  - Sie können den Code schnell mit älteren Übungen vergleichen.
  - Sie haben alles beieinander und es ist relativ einfach.
  - Sie können sich auf das Code schreiben konzentrieren.

Zusammenfassung

- Entwicklungsumgebungen sind hochspezialisierte SW-Produkte!
  - Darauf ausgelegt, Entwickler*innen die Arbeit zu erleichtern.
  - Man muss die Bedienung aber wirklich erlernen!
- Es lassen sich alle Funktionen interaktiv über Menüs und Wizards ausführen, aber: Für die wirklich effiziente Arbeit nutzt man sehr oft Shortcuts!
  - siehe z.B. https://www.shortcutworld.com/
- Damit sie effizient arbeiten können, müssen Sie die Bedienung Ihrer IDE wirklich bewusst erlernen.
- Machen Sie sich nie zu stark von einer bestimmten IDE abhängig!
- **Wichtig**: Alle Übungen, die **nicht** aus dem Buch (BlueJ) kommen, ab sofort in der IDE durchführen.

Fragen?